



Home

Library

Profile

Stories

Stats

Following

Allan Milne Lees

UX Collective

The Startup

Michael Ndubuaku

Better Marketing

Data Science Colle...

UX Planet

TheOptimizationKi...

Entrepreneurship ...

Aaron Dinin, PhD

More

CodeX

Member-only story

Everything connected
with Tech & Code
Follow to join our 1M+
monthly readers
[Follow publication](#)

Old Drugs, New Uses: the Graph Neural Network Approach

Using PyG's link prediction to find new drug indications



Sixing Huang

Follow

13 min read · May 8, 2024

Photo by [danilo.alvesd](#) on [Unsplash](#)

The diagram illustrates a network of diseases and treatments. Red circles represent diseases: Streptococcal Infections, Sinusitis, and Otitis Externa. Green circles represent treatments: chloramphenicol, ciprofloxacin, and moxifloxacin. Arrows show relationships: 'MAY BE TREATED BY' (thin black), 'Message' (thick pink), and unlabeled green arrows. A dashed line with '???' connects Sinusitis and chloramphenicol. A legend shows 'Own' (green) and 'Neighbor' (pink) features.

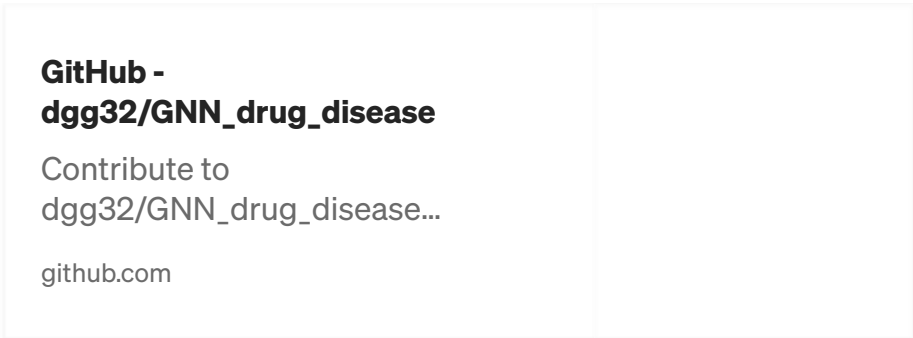
Page 2 of 29

itself and from its neighbors. This may lead to better performances.
Image by author.

GNN belongs to graph machine learning (GML), and GML is in turn a subfield of machine learning. As its name suggests, GML operates on graph data. Graphs use nodes to represent individual entities and connect these nodes with edges (Figure 1). Compared to tabular machine learning, graph machine learning can add extra information to nodes by aggregating data from their neighbors. GML can be used either to predict node properties (read my previous example [here](#)) or to recommend new connections that might exist. And drug repurposing is a good use case of the latter. Here, drugs and diseases become nodes, and existing indications act as the links between them. GML can then identify promising new connections, suggesting potential uses for existing drugs (Figure 1).

However, books and tutorials about GML and GNN are scarce. The graph platform Neo4j offers GML within its Graph Data Science library (GDS). However, it is restricted to graphs stored in Neo4j. For broader use, libraries like **PyTorch Geometric (PyG)** and **Deep Graph Library (DGL)** are better options. Following the book '[*Graph Neural Networks in Action*](#),' I experimented with drug repurposing using PyG's GraphSAGE algorithm, achieving an

ROC AUC score of 88%. This article dives into this project using GNNs. Scripts for data engineering are available on my GitHub repository here.

A screenshot of a GitHub repository link. It shows the text "GitHub - dgg32/GNN_drug_disease" in bold, followed by "Contribute to dgg32/GNN_drug_disease..." and the URL "github.com".

**GitHub -
dgg32/GNN_drug_disease**
Contribute to
dgg32/GNN_drug_disease...
github.com

And the GNN was implemented on [Google Colab](#).

1. Architecture and data

Figure 1 summarizes the core concept of the project. We built a graph with two types of nodes: diseases and drugs. Drugs were linked to their known diseases through “MAY_BE_TREATED_BY” relationships. This graph was then used to train a GraphSAGE GNN model. The trained model predicted potential new treatment applications (indications) for existing drugs. Finally, I reviewed the literature for evidence supporting the high-scoring predictions. The details are as follows.

The disease list contains 410 names. Through the NIH’s [Unified Medical Language System](#) (UMLS) API, I first collected their hierarchical categories (Figure 2, right panel). Furthermore, I retrieved their lists of drugs under the “may_be_treated_by”

relationship (Figure 2, left).

MSH

Sinusitis (D012852) Add to List

Names		UMLS Concepts		Hierarchy	
Name	Term Type	Name	CUI		
Sinusitis	MH	Sinusitis	C0037199		
Sinus Infections	ET				
Infections, Sinus	PM				
Sinusitides	PM				
Sinus Infection	PM				
Infection, Sinus	PM				

Attributes

MMR: 20190617
MN: C08.730.749
MN: C09.603.692.752
AN: general or unspecified; prefer specifics; rhinosinusitis: index RHINITIS + SINUSITIS
MDA: 19990101
MN: C01.748.749
DC: 1
MN: C08.460.692.752
DX: 19660101
AQL: BL CF CI CL CN CO DG DH DI DT EC EH EM EN EP ET GE HI IM ME MI MO NU PA PC PP PS PX RH RT SU TH UR VE VI

Relations

mapped_from	Relation	Related to
Sinusitis	mapped_from	Sinusitis

may_be_treated_by Drugs connected via "MAY_BE_TREATED_BY"

Related From	Relation	Related to
Sinusitis	may_be_treated_by	alatrofloxacin
Sinusitis	may_be_treated_by	alatrofloxacin mesylate
Sinusitis	may_be_treated_by	amoxicillin
Sinusitis	may_be_treated_by	amoxicillin anhydrous

Categories

These fifth-level categories become disease features

Medical Subject Headings
MeSH Descriptors
Topical Descriptor
Diseases (MeSH Category)
Otorhinolaryngologic Diseases
Nose Diseases
Paranasal Sinus Diseases
Sinusitis
Allergic Fungal Sinusitis
Ethmoid Sinusitis
Frontal Sinusitis
Maxillary Sinusitis
Rhinosinusitis
Sphenoid Sinusitis
Medical Subject Headings
MeSH Descriptors
Topical Descriptor
Diseases (MeSH Category)
Respiratory Tract Diseases
Respiratory Tract Infections
Sinusitis
Allergic Fungal Sinusitis
Ethmoid Sinusitis
Frontal Sinusitis
Maxillary Sinusitis
Rhinosinusitis
Sphenoid Sinusitis
Medical Subject Headings
MeSH Descriptors
Topical Descriptor
Diseases (MeSH Category)
Infections
Respiratory Tract Infections
Sinusitis
Allergic Fungal Sinusitis
Ethmoid Sinusitis
Frontal Sinusitis
Maxillary Sinusitis

Figure 2. MSH information about sinusitis. Image by author.

This first round of API calls resulted in 2,945 drugs. Again through the UMLS API, I got their mechanisms of action (MOA), parents, and physiologic effect (PE).

**UMLS**
Metathesaurus Browser

RXNORM
chloramphenicol (2348)

Names

Name	Term Type
chloramphenicol	IN

UMLS Concepts

Name	CUI
chloramphenicol	C0008168

Attributes

None.

Relations

contraindicated_with_disease

Related From	Relation	Related to
chloramphenicol	contraindicated_with_disease	Drug Hypersensitivity

has_form

Related From	Relation	Related to
chloramphenicol	has_form	chloramphenicol palmitate
chloramphenicol	has_form	chloramphenicol sodium succinate
chloramphenicol	has_form	chloramphenicol stearate
chloramphenicol	has_form	chloramphenicol succinate

has_mechanism_of_action

Related From	Relation	Related to
chloramphenicol	has_mechanism_of_action	Protein Synthesis Inhibitors

has_parent

Related From	Relation	Related to
chloramphenicol	has_parent	Amphenicol-class Antibacterial
chloramphenicol	has_parent	C [Preparations]

has_physiologic_effect

Related From	Relation	Related to
chloramphenicol	has_physiologic_effect	Decreased Protein Synthesis

Figure 3. UMLS provides mechanisms of action, parents, and physiologic effects for each drug. Image by author.

I then loaded these data into Neo4j for some preliminary analysis (Figure 4).

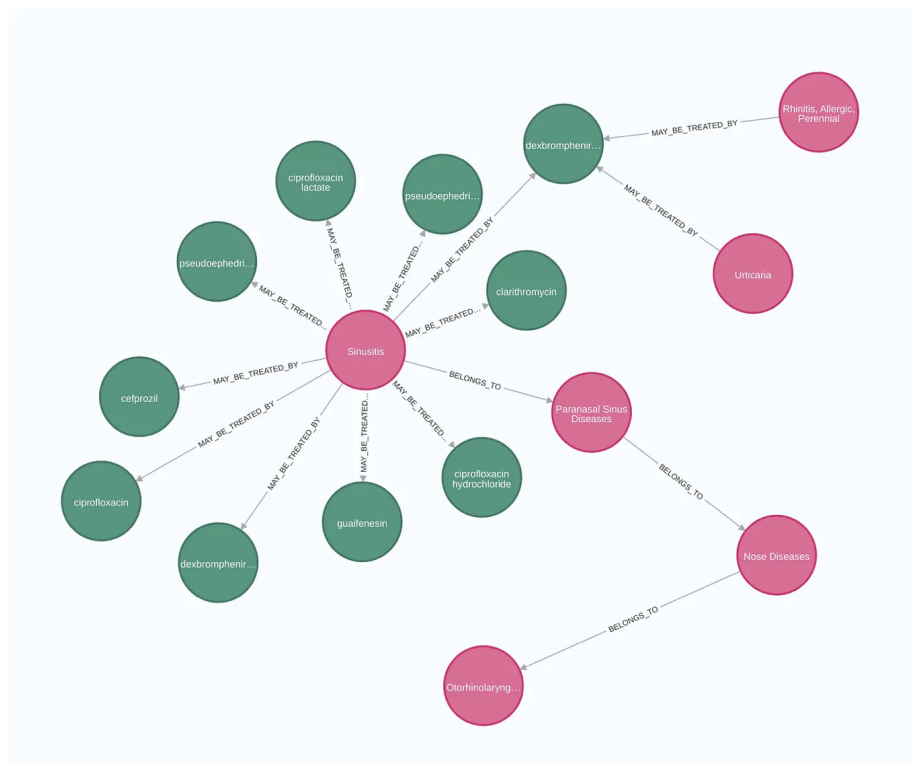


Figure 4. A subset of the drug-disease graph around Sinusitis. It shows that the drugs (green nodes) are connected to disease nodes (red nodes) via the MAY_BY_TREATED_BY relation. And diseases are connected to their categories via the BELONGS_TO relation. Image by author.

The graph structure in Figure 4 isn't suitable for PyG's link prediction task on mixed-type graphs (heterogeneous graphs). PyG requires bipartite graphs, which have two node types connected by a single relationship type. To achieve this, we eliminated the "BELONGS_TO" relation (Figure 4) by incorporating disease categories as properties of the disease nodes themselves. These "category" properties can then be used as one-hot features within the GNN model. A simple strategy is just to take the values of a certain level on the hierarchy. For example, if we select the fifth level, "Otorhinolaryngologic Diseases", "Respiratory Tract Diseases", and "Infections" would become the

values in the “category” property of sinusitis (Figure 2).

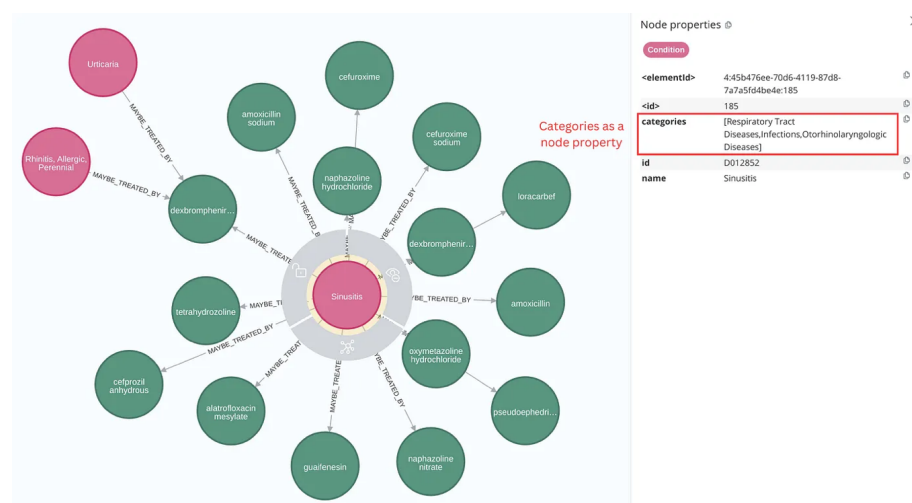


Figure 5. The restructured bipartite graph around Sinusitis. In contrast to Figure 4, this graph is bipartite that only contains two types of nodes and one type of relation. The fifth-level categories became a node property.

Image by author.

I calculated the number of categories on each level and found that the fifth level strikes a good balance. On the one hand, there are 24 different categories on that level after a cleanup. That is not too many for a one-hot feature. On the other hand, those categories remain understandable for general audiences. In comparison, drugs had MOA, parents, and PE as node properties. The pipe symbol “|” was used to separate multiple property values.

id	name	categories
D000086382	COVID-19	Respiratory Tract Diseases Infections
D014594	Uterine Neoplasms	Neoplasms Urogenital Diseases
D011349	Proctitis	Digestive System Diseases
D012859	Sjogren's Syndrome	Skin and Connective Tissue Diseases Eye Diseases Musculoskeletal Di
D013180	Sprains and Strains	Wounds and Injuries
D016889	Endometrial Neoplasms	Neoplasms Urogenital Diseases
D003324	Coronary Artery Disease	Cardiovascular Diseases
D015658	HIV Infections	Immune System Diseases Infections Urogenital Diseases Pathological Cond
D003704	Dementia	Mental Disorders Nervous System Diseases
D006816	Huntington Disease	Pathological Conditions, Signs and Symptoms Mental Disorders Nervou
D013274	Stomach Neoplasms	Digestive System Diseases Neoplasms
D008193	Lyme Disease	Infections
D003110	Colonic Neoplasms	Digestive System Diseases Neoplasms
D003554	Cystinosis	Nutritional and Metabolic Diseases Congenital, Hereditary, and Neonatal Dis
D002277	Carcinoma	Neoplasms
D008107	Liver Diseases	Digestive System Diseases
D014927	Wolff-Parkinson-White Syndrome	Cardiovascular Diseases Pathological Conditions, Signs
D012004	Rectal Neoplasms	Digestive System Diseases Neoplasms
D020529	Multiple Sclerosis, Relapsing-Remitting	Immune System Diseases Nervous System Diseases
D001919	Bradycardia	Cardiovascular Diseases Pathological Conditions, Signs and Symptoms
D010518	Periodontitis	Stomatognathic Diseases
D005185	Fallopian Tube Neoplasms	Neoplasms Urogenital Diseases

Figure 6. The disease TSV file. Image by author.

Figure 5 shows the bipartite graph after the feature conversion. It contains 2,945 drugs, 410 diseases, and 6,692 drug-disease pairs (Figure 5). They were stored as three separate TSV files. Figure 6 is a snapshot of the disease TSV file.

2. PyG

I then turned to PyG to implement a Graph Neural Network (GNN) model. PyG is a Python library for graph neural networks. You can run this whole PyG project on a GPU in Google Colab. I uploaded the three TSV files to Google Drive and then exposed them to Colab via the Mount Drive button. In Code 1, I loaded the files in pandas data frames.

```
## Code 1
drug_path = '/content/drive/MyDrive/drug_link_predic
disease_path = '/content/drive/MyDrive/drug_link_pre
disease_drug_path = '/content/drive/MyDrive/drug_lin
```

```

drug_df = pd.read_csv(drug_path, index_col='RXNORM_i
disease_df = pd.read_csv(disease_path, index_col='id
treatment_df = pd.read_csv(disease_drug_path, sep="\

```

While real-world graphs come in many shapes and sizes, they all can be handled by PyG when transformed into clean, pure mathematical matrices. To get the data into PyG formats, I one-hot encoded the node features and also assigned new numerical identifiers (re-indexed) to each node. Finally, I rebuilt the edges based on these new identifiers.

```

## Code 2
##### One-hot encode the node features
# Split moa and convert into indicator variables:
moa = drug_df['moa'].str.get_dummies('|')
# Use moa as drug input features:
drug_feat = torch.from_numpy(moa.values).to(torch.fl

# Split disease_categories and convert into indicato
disease_categories = disease_df['categories'].str.ge
# Use disease_categories as drug input features:
disease_feat = torch.from_numpy(disease_categories.v

##### re-index the nodes incrementally
# Create a mapping from unique user indices to range
unique_disease_id = treatment_df['from'].unique()
unique_disease_id = pd.DataFrame(data={
    'diseaseId': disease_df.index,
    'mappedID': pd.RangeIndex(len(disease_df)),
})

# Create a mapping from unique movie indices to rang
unique_drug_id = treatment_df['to'].unique()
unique_drug_id = pd.DataFrame(data={
    'drugId': drug_df.index,
    'mappedID': pd.RangeIndex(len(drug_df)),

```

```

})

##### reformat the edge with the new node indices
# Perform merge to obtain the edges from drugs and d
treatment_disease_id = pd.merge(treatment_df['from'],
                                left_on='from', right_on=
treatment_disease_id = torch.from_numpy(treatment_di
treatment_drug_id = pd.merge(treatment_df['to'], uni
                                left_on='to', right_on=
treatment_drug_id = torch.from_numpy(treatment_drug_

# With this, we are ready to construct our `edge_index`
# following PyG semantics:
edge_index_disease_to_drug = torch.stack([treatment_
print (edge_index_disease_to_drug.size())

```

This particular run relied solely on the Mechanism of Action (MOA) as the drug feature, because MOA offered a much more manageable number of features (416) compared to the ‘parents’ and ‘PE’ features. After the conversion (Code 2), the node and edge matrices look like this.

1. Mapping of disease IDs to consecutive values:

=====

	diseaseId	mappedID
0	D009889	0
1	D008224	1
2	D014009	2
3	D012559	3
4	D013585	4

2. The disease feature matrix:

=====

	Infections	Hemic and Lymphatic Diseases	.
id			
D009889	1		0
D008224	0		1
D014009	1		0
D012559	0		0
D013585	0		0

3. Mapping of drug IDs to consecutive values:

=====

	drugId	mappedID
0	1363268	0
1	1363267	1
2	1603296	2
3	1604346	3
4	2534233	4

4. Final edge indices pointing from disease to drug:

=====

```
tensor([[ 341,  301,  272, ...,  282,  225,   72],
        [1953,   84, 2264, ..., 2469, 1140, 1162]])
```

These matrices were then packaged as a PyG HeteroData object. It is noteworthy that the x attribute holds the node features (Code 3).

```
## Code 3
##### Pacakge the data into a HeteroData object

from torch_geometric.data import HeteroData
import torch_geometric.transforms as T

data = HeteroData()

# Save node indices:
data["disease"].node_id = torch.arange(len(unique_di
data["drug"].node_id = torch.arange(len(unique_drug_

# Add the node features and edge indices:
data["disease"].x = disease_feat
data["drug"].x = drug_feat
data["disease", "may_be_treated_by", "drug"].edge_in

# We also need to make sure to add the reverse edges
# in order to let a GNN be able to pass messages in
# We can leverage the `T.ToUndirected()` transform f
data = T.ToUndirected()(data)
```

Afterward, I employed an 80:10:10 data split, allocating 80% of the links for training the model, 10% for validation during training, and the remaining 10% for final testing after training is complete.

Code 4

```
# For this, we first split the set of edges into
# training (80%), validation (10%), and testing edge
# Across the training edges, we use 70% of edges for
# and 30% of edges for supervision.
# We further want to generate fixed negative edges f
# Negative edges during training will be generated o
# We can leverage the `RandomLinkSplit()` transform
transform = T.RandomLinkSplit(
    num_val=0.1,
    num_test=0.1,
    disjoint_train_ratio=0.3,
    neg_sampling_ratio=2.0,
    add_negative_train_samples=False,
    edge_types=('disease', 'may_be_treated_by', 'dru
    rev_edge_types=('drug', 'rev_may_be_treated_by',
)

train_data, val_data, test_data = transform(data)
```

Both `disjoint_train_ratio=0.3` and `neg_sampling_ratio=2.0` are unique to GNN. The former indicated a 70/30 split of the graph edges. 70% were used for message passing and 30% were reserved for supervision. In other words, messages were only allowed to pass among those 70% of edges, but not among the 30%. Those 30% were used as learning targets. The negative sampling

ratio dictates that for each existing edge, PyG created two non-existing edges as negative targets.

```
## Code 5
# In the first hop, we sample at most 20 neighbors.
# In the second hop, we sample at most 10 neighbors.
# In addition, during training, we want to sample ne
# a ratio of 2:1.
# We can make use of the `loader.LinkNeighborLoader`
from torch_geometric.loader import LinkNeighborLoader

# Define seed edges:
edge_label_index = train_data['disease', 'may_be_tre
edge_label = train_data['disease', 'may_be_treated_b

train_loader = LinkNeighborLoader(
    data=train_data,
    num_neighbors=[20, 10],
    neg_sampling_ratio=2.0,
    edge_label_index= (('disease', 'may_be_treated_by
    edge_label=edge_label,
    batch_size=128,
    shuffle=True,
)
```

In the next code block (Code 5), the edge_label_index variable holds the supervision edges (30%) while edge_label holds the message passing edges (70%). Similar to other neural networks, GNN are also trained in batches. PyG simplifies batch handling through its

LinkNeighborLoader class. In Code 5, we set the batch size to 128. The num_neighbors parameter merits some explanation. A core strength of GNNs is their ability to aggregate information between neighboring nodes (message passing). However,

considering messages from nodes several hops away can lead to a significant amount of computation. To address this, we can limit the number of neighbors considered during message passing. This is achieved by setting the `num_neighbors` parameter. In our example, a value of `[20, 10]` indicates that at most 20 neighbors will be sampled in the first message passing step (1-hop), and at most 10 neighbors will be sampled in the second step (2-hop). This helps control the computational cost.

The model is defined in Code 6.

```
## Code 6
from torch_geometric.nn import SAGEConv, to_hetero
import torch.nn.functional as F

class GNN(torch.nn.Module):
    def __init__(self, hidden_channels):
        super().__init__()

        self.conv1 = SAGEConv(hidden_channels, hidden_channels)
        self.conv2 = SAGEConv(hidden_channels, hidden_channels)

    def forward(self, x: Tensor, edge_index: Tensor):
        x = F.relu(self.conv1(x, edge_index))
        x = self.conv2(x, edge_index)
        return x

# Our final classifier applies the dot-product between
# node embeddings to derive edge-level predictions:
class Classifier(torch.nn.Module):
    def forward(self, x_disease: Tensor, x_drug: Tensor):
        # Convert node embeddings to edge-level representations
        edge_feat_disease = x_disease[edge_label_index[0]]
        edge_feat_drug = x_drug[edge_label_index[1]]

        # Apply dot-product to get a prediction per edge
```

```

        return (edge_feat_disease * edge_feat_drug).

class Model(torch.nn.Module):
    def __init__(self, hidden_channels):
        super().__init__()
        self.disease_lin = torch.nn.Linear(data["dis
        self.drug_lin = torch.nn.Linear(data["drug"]
        self.disease_emb = torch.nn.Embedding(data["
        self.drug_emb = torch.nn.Embedding(data["dru

        # Instantiate homogeneous GNN:
        self.gnn = GNN(hidden_channels)

        # Convert GNN model into a heterogeneous var
        self.gnn = to_hetero(self.gnn, metadata=data

        self.classifier = Classifier()

    def forward(self, data: HeteroData) -> Tensor:
        x_dict = {
            "disease": self.disease_lin(data["disease"
            "drug": self.drug_lin(data["drug"].x) + se
        }

        # `x_dict` holds feature matrices of all nod
        # `edge_index_dict` holds all edge indices o
        x_dict = self.gnn(x_dict, data.edge_index_di
        pred = self.classifier(
            x_dict["disease"],
            x_dict["drug"],
            data['disease', 'may_be_treated_by', 'dr
        )

        return pred

model = Model(hidden_channels=64)

```

The Model class, through its forward function, orchestrates how data flows within the network. First, both nodes and their associated features are transformed: nodes are embedded in a lower-dimensional space, while features are processed by

a linear layer. These transformed representations are then combined and fed into two stacked SAGEConv layers. These layers perform message passing, allowing the network to learn from the first two hops of neighboring nodes (70% of the edges). Finally, a classifier layer takes the dot product of the learned embeddings for drugs and diseases and uses the output logits to predict the supervision edges (30% of the edges).

Training in PyG follows the standard PyTorch training loop structure. The binary cross entropy is calculated as the loss (Code 7).

```
## Code 7
import tqdm
import torch.nn.functional as F

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Device: '{device}'")

model = model.to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)

for epoch in range(1, 20):
    total_loss = total_examples = 0
    for sampled_data in tqdm.tqdm(train_loader):
        optimizer.zero_grad()

        sampled_data.to(device)
        pred = model(sampled_data)

        ground_truth = sampled_data['disease', 'may_disease']
        loss = F.binary_cross_entropy_with_logits(pred, ground_truth)

        loss.backward()
        optimizer.step()
        total_loss += float(loss) * pred.numel()
        total_examples += pred.numel()
```

```
print(f"Epoch: {epoch:03d}, Loss: {total_loss /
```

The model was trained after 20 epochs. It was then validated and tested. While only the validation loop is shown in Code 8, swapping the validation data for the test data within the code effectively creates the test loop.

```
## Code 8: the validation loop. If we change the dat
## we got ourselves the test loop

from sklearn.metrics import roc_auc_score

# change these two lines to test_data
edge_label_index = val_data['disease', 'may_be_treat
edge_label = val_data['disease', 'may_be_treated_by'

val_loader = LinkNeighborLoader(
# change this line to test_data
    data=val_data,
    num_neighbors=[20, 10],
    edge_label_index= (('disease', 'may_be_treated_by'
    edge_label=edge_label,
    batch_size=3 * 128,
    shuffle=False,
)

preds = []
ground_truths = []
for sampled_data in tqdm.tqdm(val_loader):
    with torch.no_grad():
        sampled_data.to(device)
        preds.append(model(sampled_data))
        ground_truths.append(sampled_data['disease',

pred = torch.cat(preds, dim=0).cpu().numpy()
ground_truth = torch.cat(ground_truths, dim=0).cpu()
auc = roc_auc_score(ground_truth, pred)
print()
print(f"Validation AUC: {auc:.4f}")

## Validation AUC: 0.8826
```

```
## Test AUC: 0.9029
```

Figure 7. The ROC curve of the validation run. Image by author.

The validation and test loops returned ROC AUC scores of 88% and 90%, respectively (Figure 7). Finally, we could convert the logits from the classifier to probabilities like this.

```
## Code 9
def logit_to_proba(logit):
    return 1/(1+ math.exp(-logit))
```

With this conversion, we could see how confident the model was about each prediction.

3. Examining some novel drug-disease predictions

The validation and test scores indicated decent model performances. Interestingly, the model made some novel drug-disease connections during my multiple runs. Here are some of them (Table 1).

Table 1. The novel drug-disease connections that were predicted during multiple GNN test runs.

The first pair Pneumonia, Bacterial-cefoxitin sodium received a high prediction score. Cefoxitin sodium is a beta-lactam antibiotic with bactericidal activity. So this new link seems reasonable. In fact, medlineplus.gov indeed stated that *cefoxitin*

injection is used to treat infections caused by bacteria, including pneumonia and other lower respiratory tract (lung) infections.

There is also evidence supporting the Sinusitis-chloramphenicol prediction. Based on their own clinical experience and a pilot study, Nielsen *et al.* stated in 2013 that chloramphenicol eye drops may have a favorable effect on many patients troubled by acute maxillary sinusitis.

In contrast, Listeriosis-amphotericin B liposomal can be a harmful combination. Geginat *et al.* observed that the amphotericin B weakened the acquired immune protection against *Listeria* in mice. Specifically, the drug inhibited the CD8 T cell function. Regarding the fourth pair, liothyronine is used to add thyroid hormone to the body for patients with thyroid problems. However, there's no evidence to suggest that it's effective in managing muscle spasticity.

Conclusion

Computer-aided and data-driven drug developments have seen a surge thanks to machine learning and big data. It allows researchers to analyze massive datasets on molecules and their

properties. In this article, I described my experiment with graph neural networks to predict novel drug indications. The GNN was implemented in PyG and achieved good validation and test scores. Nevertheless, users must examine its link prediction critically. As reported above, some of the predicted drug-disease connections seemed promising, while the others should be avoided.

PyG allows us to customize their own GNN. However, this requires writing low-level code that might prove difficult even for seasoned programmers. However, the current PyG pipeline requires a bipartite graph format, forcing us to squeeze information like Mechanism of Action (MOA), physiological effects, and drug relationships into single node properties. This data engineering leads to loss of information and likely sacrifices valuable insights, because the compressed format might obscure the intricate connections among drugs. In comparison, Neo4j's GDS could be more friendly for GNN newcomers.

There are many ways to extend the current study. For starters, we could try to use "PE" and "parents" as drug properties. Or we can combine all three of them. In any case, some dimension reductions will be needed, because "PE" has 509 and "parents" has 404 unique values. In addition, we can get more drug-disease pairs from UMLS. We can also try

other architectures such as graph attention networks (GAT) on the same task. Last but not least, we could compare the performance of GNN with other machine learning methods to understand its strengths and weaknesses.

Let's learn together!

Edit description

dgg32.medium.com

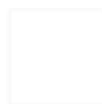
Pytorch

Healthcare

Drug Repurposing

Graph Neural Networks

Neo4j



Published in CodeX

29K followers · Last published 1 hour ago

Follow

Everything connected with Tech & Code. Follow to join our 1M+ monthly readers



Written by Sixing Huang

1.5K followers · 131 following

Follow



A Neo4j Ninja, German bioinformatician in Gemini Data. I like to try things: Cloud, ML, satellite imagery, Japanese, plants, and travel the world.

No responses yet



Paul Beglin

What are your thoughts?

More from Sixing Huang and CodeX

 Sixing Huang

Graph Visualization for DuckPGQ

Using Neo4j Visualization Library to visualize a drug...

★ Apr 9 🖱 42 💬 2 📌⁺ ⋮

 In CodeX by Daniel Boadzie

Building Real-Time Dashboards with...

Welcome Back, Dashboard Builders!

★ Aug 18 🖱 420 💬 1 📌⁺ ⋮

 In CodeX by AI Rabbit

Supercharge Your n8n Automations with...

Apify has just launched a native node for n8n,...

★ Aug 28 🖱 97 💬 1 📌⁺ ⋮

 Sixing Huang

Building a Stock-Picker with CrewAI

Screen and evaluate stocks with AI agents

★ Aug 13 🖱 13 📌⁺ ⋮

See all from Sixing Huang

See all from CodeX

Recommended from Medium



In Neo4j Develo... by Alex Gil...

Knowledge Graph Generation

Strategies and methods for knowledge graph generation

Sep 22



88



2



In Level Up Co... by Fareed K...

Building an Advanced Agentic RAG Pipeline...

Ambiguity Checks, Multi-Tool Planning, Self-Correction,...



Sep 16



1.2K



18





Sakshee Patil

I never understood the fuss over using...

Is it worth the hype? How knowledge graphs (KGs) are...

Sep
12



56



1



Gradient Explosion

In-Depth Analysis of the Latest Deep...

I. Introduction

Sep 16



Calogero Kalos Bonasia

The hidden world of knowledge graphs: a...

Imagine entering a large library. Not just any library,...

Sep 6



110



1



Tuhin Sharma

Fast GraphRAG Architecture: Speed,...

Fast GraphRAG is built on the principle that memory...

Apr
20



41



1

[See more recommendations](#)

[Help](#) [Status](#) [About](#) [Careers](#) [Press](#) [Blog](#) [Privacy](#) [Rules](#)
[Terms](#) [Text to speech](#)